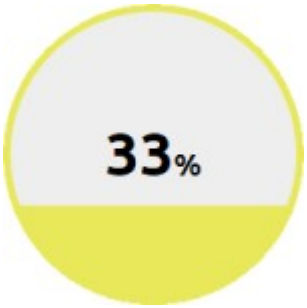
Componente para mostrar un porcentaje usando Angular y clip-path.

Introducción

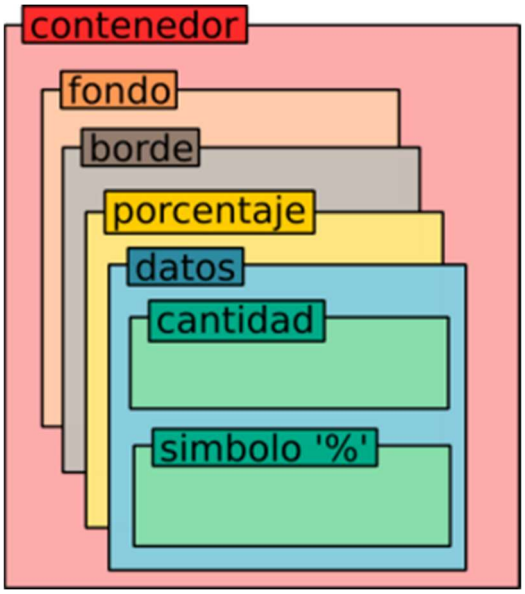
Clip-path es una propiedad de CSS que permite mostrar una parte del elemento al que se le aplique. La forma mediante la que se mostrará el contenido del elemento se puede definir a partir de formas geométricas o trazos SVG.

Vamos a usar esta propiedad de CSS para crear un circulo lleno hasta un valor porcentual que se pasará como parámetro. Por ejemplo, para valores del 33% y 75% se vería así:





El componente estará formado por un elemento que hará de contenedor, que contendrá cuatro elementos más que harán las funciones de mostrar el fondo, el borde, el relleno del círculo y el número con el porcentaje a mostrar.



Para rellenar el circulo según el porcentaje que se reciba como parámetro se usará la propiedad inset de clip-path. A esta propiedad se le pueden indicar 4 valores que servirán para definir el recorte superior, derecho, inferior e izquierdo. Por ejemplo:

```
clip-path: inset (100px 20px 0 0)
```

Para este componente usaremos esta regla dando valor solamente al recorte superior.

El componente

Creamos el componente mediante el comando:

```
ng g c components/percentageCicle
```

Parámetros

Añadimos los Inputs para los parámetros que recibirá el componente.

size: el tamaño, o diámetro, del círculo.

```
@Input('size') set size(value: number) {
          if (value !== undefined &&
                    !Number.isNaN(Number.parseFloat(value as any)) &&
                    value > 0) {
                    this.pSize = value;
          }
}
```

con una función set se comprueba que el valor recibido es un número, antes de asignarlo a la propiedad privada del componente que lo usará.

percentage: el porcentaje a mostrar dentro del círculo.

```
@Input('percentage') set percentage(value: number) {
          if (Number.isNaN(Number.parseFloat(value as any))) {
                    this.pPercent = 0;
          } else if (value < 0) {
                    this.pPercent = 0;
          } else if (value > 100) {
                    this.pPercent = 100;
          } else {
                    this.pPercent = value;
          }
}
```

igual que el parámetro **size**, se comprueba que sea un número válido y que se encuentra dentro de unos límites.

animate: si ha de mostrar o no una animación con el elemento que se usa para rellenar el círculo con el valor del porcentaje. La animación va del valor 0 al que se haya indicado.

```
@Input() animate: boolean;
```

showNumber: si ha de mostrar o no el número con el valor del porcentaje en el centro del círculo.

```
@Input() showNumber: boolean;
```

backgroundImage: url de la imagen a usar como fondo del círculo.

```
@Input() backgroundImage: string;
```

backgroundFilter: si ha de aplicar o no filtros al fondo del círculo. Se aplican escala de grises y desenfoque.

```
@Input() backgroundFilter: boolean;
```

percentColor: color a usar para el relleno del círculo para el porcentaje. También se usará para el color del borde del círculo.

```
@Input('percentColor') set percentColor(value: string) {
          if (isValidColor(value)) {
                    this.pPercentColor = value;
          }
}
```

backgroundColor: color a usar para el fondo del círculo. Se mostrará en caso de que no se indique una imagen para el fondo.

```
@Input('backgroundColor') set backgroundColor(value: string) {
          if (isValidColor(value)) {
                    this.pBackgroundColor = value;
          }
}
```

percentTextColor: color a usar para el texto del centro del círculo que muestra el valor del porcentaje.

```
@Input('percentTextColor') set percentTextColor(value: string) {
          if (isValidColor(value)) {
                    this.pPercentTextColor = value;
          }
}
```

Los colores se comprueban que sean válidos mediante la siguiente función:

```
function isValidColor(color: string): boolean {
          const element: HTMLElement = document.createElement('div');
          element.style.fill = color;
          return (element.style.fill !== '');
}
```

Esta función recibe un color, en cualquier formato válido (‘red’, ‘#fff’, ‘#fa1233’, ...), crea un elemento div e intenta asignarle el color. Devuelve true o false, dependiendo si el color se pudo asignar a no al elemento creado.

Propiedades privadas

Creamos una serie de propiedades privadas con los valores que se comprobarán cuando se reciban como parámetros y el que definirá el recorte que se aplicará al elemento que rellena el círculo. También sirven para que queden definidos algunos valores por defecto.

```
private pSize = 200;
private pPercent: number;
private pInset: string;
private pPercentColor = '#E8E85A';
private pBackgroundColor = '#EEEEEE';
private pPercentTextColor = '#000000';
```

Getters

Para usar las propiedades privadas y alguna que requiere de cierta lógica en la plantilla HTML usamos funciones get.

Para los colores:

```
get percentColor(): string {
          return this.pPercentColor;
}
get backgroundColor(): string {
          return this.pBackgroundColor;
}
get percentTextColor(): string {
          return this.pPercentTextColor;
}
```

Para el tamaño del círculo añadimos al final la unidad de medida, en este caso píxeles:

```
get sizeStyle(): string {
          return `${this.pSize}px`;
}
```

Para el tamaño del texto del número a mostrar en el centro del círculo y el símbolo del '%', también añadimos la unidad de medida, pero reduciendo su tamaño un 20 y un 10 por ciento respecto al tamaño del círculo:

```
get numberAmountSize(): string {
          return `${this.pSize * .20}px`;
}
get numberPercentageSize(): string {
          return `${this.pSize * .10}px`;
}
```

Para la imagen de fondo, dependiendo si se ha recibido o no el parámetro, devolverá la regla CSS para establecer al imagen de fondo con el valor recibido o una cadena en blanco:

```
get backgroundImageStyle(): string {
          if (this.backgroundImage) {
                    return `url(${this.backgroundImage})`;
          }
          return '';
}
```

Para animar o no el relleno del círculo estableceremos el valor de transition-duration de CSS a 1 o 0 segundos según lo haya indicado el usuario mediante el parámetro **animate** o usando el valor por defecto:

```
get transitionTime(): string {
          return this.animate ? '1s' : '0s';
}
```

El porcentaje devuelve la propiedad privada que se contendrá el valor del parámetro **percentage**:

```
get percentColor(): string {
          return this.pPercentColor;
}
```

El recorte a aplicar el elemento:

```
get inset(): any {
          return this.pInset;
}
```

Funcionamiento

El componente ejecuta, según convenga, un método que establece el recorte a aplicar en el elemento que rellena el círculo. El método que realiza el recorte es el siguiente:

```
private setInset(value?: number): void {
          if (value === undefined) {
                    value = this.pPercent;
          }
          this.pInset = `inset(${100 - value}% 0 0 0)`;
}
```

Puede recibir un parámetro opcional, que sería el porcentaje a mostrar en el círculo. Si no lo recibe, usa el valor guardado en **pPercent**. El recorte será la parte que no se mostrará del elemento, por lo que es la diferencia entre el valor del porcentaje y el 100%. Este método, finalmente, devuelve la regla CSS necesaria para recortar el elemento por la parte superior esa diferencia.

Este método se llamará en tres situaciónes:

Al iniciar el componente, pasándole un valor de 0 para que el recorte sea total y se muestre un 0% de relleno del círculo.

```
ngOnInit(): void {
          this.setInset(0);
}
```

Una vez se haya iniciado la vista del componente, llamamos al método sin valor para que use el que se ha guardado en **pPercent**. Se llama al método dentro de un *setTimeout* para evitar que se produzca un error al comprobar Angular que el valor a cambiado durante el ciclo. Al encapsular la llamada dentro del *setTimeout* se asegura que se ejecutara cuando la pila esté vacía y Angular haya finalizado sus comprobaciones.

```
ngAfterViewInit(): void {
          setTimeout(() => {
                    this.setInset();
          }, 0);
}
```

Y por último, cada vez que el valor de **percentage** cambie. Mediante el hook *onChanges* se vigila el valor y si cambia sin que sea su primer cambio (este se produce la primera vez que se establece), se guarda el valor en la propiedad **pPercent** y se llama al método.

```
ngOnChanges(changes: SimpleChanges): void {
          if (changes.percentage && !changes.percentage.firstChange) {
                    this.pPercent = changes.percentage.currentValue;
                    this.setInset();
          }
}
```

Plantilla HTML

El contenido de la plantilla HTML para el componente sería el siguiente:

```
<div class="circle__wrapper" [style.width]="sizeStyle" [style.height]="sizeStyle">
          <div class="circle__background"
                    [ngClass]="{'backgroundFilter':backgroundFilter}"
                    [style.backgroundImage]="backgroundImageStyle"
                    [style.backgroundColor]="backgroundColor"></div>
          <div class="circle__border" [style.borderColor]="percentColor"></div>
          <div class="circle__percentage"
                    [style.borderColor]="percentColor"
                    [style.backgroundColor]="percentColor"
                    [style.clip-path]="inset"
                    [style.transition-duration]="transitionTime"></div>
          <ng-container *ngIf="showNumber">
                    <div class="circle__number">
                              <div [style.color]="percentTextColor">
                                        <span
                                                  class="circle__number-amount"
                                                  [style.fontSize]="numberAmountSize">{{
percent }}</span>
                                        <span
                                                  class="circle__number-percentage"
                                        [style.fontSize]="numberPercentageSize">%</span>
                              </div>
                    </div>
          </ng-container>
</div>
```

Tenemos el elemento de clase **circle__wrapper** que recibe el tamaño definido mediante parámetro.

El elemento de clase **circle__background** que aplicará la clase *backgroundFilter* para el degradado y el desenfoque si fuera necesario y los estilos para la imagen y color de fondo.

El elemento de clase **circle__border** que recibe el color del porcentaje para el borde.

El elemento de clase **circle__percentage** que recibe el color del porcentaje, el recorte a realizar y el tiempo a aplicar a la transición.

El elemento de clase **circle_number** que recibe el color para el texto, conteniendo los elementos de clase **circle__number-amount** y **circle__number-percentage**, que reciben sus respectivos tamaños para la fuente.

Hoja de estilos (SCSS)

La hoja de estilos que usa el componente es la siguiente:

```
@mixin circle-radius {
	border-width: 5px;
	border-style: solid;
	border-radius: 100%;
}
@mixin circle-position {
	position: absolute;
	left: 0;
	top: 0;
}
@mixin circle-size {
	width: 100%;
	height: 100%;
}
.circle {
	&__wrapper {
		position:relative;
	}
	&__background,
	&__border,
	&__percentage {
		box-sizing: border-box;
		@include circle-position();
		@include circle-size();
	}
	&__border,
	&__percentage {
		@include circle-radius();
	}
	&__background {
		clip-path: circle(50% at 50% 50%);
		background-position: center center;
		background-attachment: scroll;
		background-size: cover;
		background-repeat: no-repeat;
	}
	&__percentage {
		transition-property: all;
		transition-timing-function: ease-in-out;
	}
	&__number {
		@include circle-position();
		@include circle-size();
		display: flex;
		align-items: center;
		justify-content: center;
		font-family: UbuntuBold;
		& > div {
			display: flex;
			align-items: baseline;
		}
	}
}
.backgroundFilter {
	-moz-filter: blur(2px) grayscale(1);
	filter: blur(2px) grayscale(1);
}
```

En el fichero styles.scss, en la raíz de la aplicación, se cargan las fuentes necesarias.

```
@font-face {
          font-family: "RobotoRegular";
          src: url(assets/fonts/Roboto/Roboto-Regular.ttf) format("truetype");
}
@font-face {
          font-family: "UbuntuBold";
          src: url(assets/fonts/Ubuntu/Ubuntu-Bold.ttf) format("truetype");
}
```